

Machine Learning

A 4-page quick reference guide

1. Core Concepts

Machine learning builds systems that learn patterns from data rather than following explicit rules. The three broad paradigms are supervised, unsupervised, and reinforcement learning.

Types of Learning

- **Supervised learning** — learns from labeled data (regression, classification)
- **Unsupervised learning** — finds structure in unlabeled data (clustering, dimensionality reduction)
- **Reinforcement learning** — an agent learns via rewards from interacting with an environment

The ML Workflow

- Collect and clean data
- Split into train / validation / test sets
- Choose a model and train it on the training set
- Tune hyperparameters using the validation set
- Evaluate final performance on the held-out test set

2. Model Training Basics

Training minimizes a loss function that measures the gap between predictions and true labels, typically using gradient-based optimization.

Example: scikit-learn Workflow

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = LogisticRegression()
model.fit(X_train, y_train)
preds = model.predict(X_test)
print(accuracy_score(y_test, preds))
```

Overfitting vs. Underfitting

- **Overfitting** — model memorizes training data, performs poorly on new data
- **Underfitting** — model is too simple to capture patterns in the data
- **Regularization** (L1/L2) — penalizes large weights to reduce overfitting
- **Cross-validation** — repeatedly splits data to get a robust performance estimate

3. Common Algorithms

Regression & Classification

- **Linear / Logistic Regression** — simple, interpretable baseline models
- **Decision Trees & Random Forests** — rule-based models, handle non-linearity well
- **Support Vector Machines** — find a maximum-margin separating boundary
- **k-Nearest Neighbors** — classifies based on closest labeled examples
- **Gradient Boosting (XGBoost, LightGBM)** — ensembles of trees, strong on tabular data

Neural Networks

Neural networks stack layers of weighted connections and non-linear activations (ReLU, sigmoid) to approximate complex functions. Deep learning refers to networks with many such layers.

```
import torch.nn as nn

class SimpleNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(784, 128)
        self.fc2 = nn.Linear(128, 10)

    def forward(self, x):
        x = torch.relu(self.fc1(x))
        return self.fc2(x)
```

4. Evaluation & Best Practices

Evaluation Metrics

- **Accuracy** — fraction of correct predictions (best for balanced classes)
- **Precision & Recall** — trade-off between false positives and false negatives
- **F1 Score** — harmonic mean of precision and recall
- **RMSE / MAE** — common regression error metrics
- **ROC-AUC** — measures ranking quality across classification thresholds

Feature Engineering

Good features often matter more than model choice. Common steps include scaling numeric features, encoding categorical variables, and handling missing values.

Best Practices

- Always hold out a test set the model never sees during training
- Watch for data leakage between train and test splits
- Start with a simple baseline model before adding complexity
- Track experiments and hyperparameters for reproducibility
- Monitor deployed models for data drift over time